MyCodeSchool

Primii pași în Python

Fișa 1 · print, tipuri de date, operatori, if / elif / else

Ai voie să folosești: variabile; tipurile `int`, `float`, `str`, `bool` și `type()`; `print` (cu virgule, `sep=` / `end=`, `\n` în text, f-string); `input` cu `int()` / `float()`; `+ - * / // %`; `== != < <= > >=`; `and or not`; `if / elif / else`; `def nume():` fără parametri și fără `return`.
Reține: o problemă = o funcție în `tipuri_operatori_if.py`, cu numele din enunț. Rulezi `main.py` pe toate exemplele; ce afișează trebuie să fie identic cu ieșirea așteptată, inclusiv mesajele de la `input`. Fișierul `.py` are numele fișei. În blocuri, **verde** = ce tastezi tu.
Încă NU folosim: funcții cu parametri sau cu `return`, `while`, `for`, `range`, liste.

Înainte de problema 1 — proiectul tău

Proiectul are două fișiere. În `tipuri_operatori_if.py` scrii problemele: `def afis_cafenea():` dă un nume codului unei probleme, iar tot ce e indentat cu 4 spații sub el face parte din ea. Codul dintr-un `def` nu pornește singur, ci doar când îl chemi: `afis_cafenea()`.

Programul pornește din `main.py`: aduce funcția din `tipuri_operatori_if.py` cu `from tipuri_operatori_if import ...` și o cheamă în `main()`. Scrie exact asta, apoi înlocuiește `pass` (= „încă nimic aici") cu pașii problemei 1.

tipuri_operatori_if.py

```
def afis_cafenea():
    pass
```

main.py

```
from tipuri_operatori_if import afis_cafenea


def main():
    afis_cafenea()


if __name__ == "__main__":
    main()
```

Cum rulezi în PyCharm: fișierele le creezi cu click dreapta pe proiect → **New → Python File** → `tipuri_operatori_if`, apoi `main` (fără `.py`). Rulezi mereu `main.py`: ▶ verde de lângă `if __name__ == "__main__":`, sau click dreapta în `main.py` → **Run 'main'** (Windows: `Ctrl+Shift+F10` · Mac: `Ctrl+Shift+R`). Rezultatul apare jos, în fereastra **Run**; acolo tastezi și valorile cerute de `input`, apoi `Enter`. La fiecare problemă nouă: adaugi funcția ei sub celelalte în `tipuri_operatori_if.py`, îi pui numele în `import` (`from tipuri_operatori_if import afis_cafenea, meniu`), schimbi apelul din `main()` și rulezi iar `main.py`.

Partea 0 — încălzire cu print

Fără `input` și fără `if`: doar `print`. Rulezi după fiecare pas și compari cu ieșirea așteptată.

1. Afișul cafenelei

print

În `tipuri_operatori_if.py`: `def afis_cafenea():`

Fiecare `print` scrie un rând. Textul stă între ghilimele și apare pe ecran fără ele.

1. Afișează `Cafeneaua din colt`: `print("Cafeneaua din colt")`.
2. Cu încă un `print`, afișează pe rândul următor `Deschis: Luni – Vineri`.
3. Lasă un rând liber cu `print()`, fără nimic între paranteze.
4. La final afișează `Azi: cafea proaspata!`.

Ieșire așteptată

```
Cafeneaua din colt
Deschis: Luni – Vineri

Azi: cafea proaspata!
```

2. Meniul

print cu virgulă

În `tipuri_operatori_if.py`: `def meniu():`

Între paranteze poți pune mai multe valori, despărțite prin virgulă: text între ghilimele și numere fără ghilimele. Python le scrie pe același rând și pune singur **un spațiu** între ele.

1. Titlul, din trei valori: `print("Meniu:", 3, "produse")`.
2. Câte un rând pe produs, cu numele, prețul și `"lei"`: `"Cafea"` cu `12`, `"Ceai"` cu `8.5`, `"Apa"` cu `5`. Prețul e număr: `12`, nu `"12"`.

Ieșire așteptată

```
Meniu: 3 produse
Cafea 12 lei
Ceai 8.5 lei
Apa 5 lei
```

3. Catalogul

sep, end, \n

În `tipuri_operatori_if.py`: `def catalog():`

`sep` alege ce se pune între valori în locul spațiului, iar `end` alege ce se pune la sfârșit în locul trecerii pe rândul următor. `\n` scris în text înseamnă „rând nou”.

1. Titlul pe două rânduri, cu un `print`: `print("Catalog\nClasa a IX-a")`.
2. `"Data:"` cu `end=" "`, ca `print`-ul următor să continue rândul.
3. Data, cu punct între numere: `print(22, 10, 2026, sep=".")`.
4. Capul de tabel `"Nume", "Mate", "Info"` cu `sep=" | "`, apoi la fel `"Ana", 9, 10` și `"Dan", 7, 8`.

Ieșire așteptată

```
Catalog
Clasa a IX-a
Data: 22.10.2026
Nume | Mate | Info
Ana | 9 | 10
Dan | 7 | 8
```

4. Bonul

text sau calcul

În `tipuri_operatori_if.py`: `def bon_cafenea():`

Între ghilimele, Python afișează textul exact cum e scris. Fără ghilimele, întâi **calculează** expresia, apoi afișează rezultatul.

1. `print("2 + 3")`, apoi `print(2 + 3)`: compară rândurile.
2. Marginea `"+----------------+"` (17 liniuțe), apoi `print("| Cafea x2", 2 * 12, "|")`. Numeri spațiile, ca bara din dreapta să cadă sub `+`.
3. La fel, 3 croissante a 7 lei (`3 * 7`), apoi marginea.
4. Totalul: `"  Total:"` (două spații în față), `2 * 12 + 3 * 7`, `"lei"`.

Ieșire așteptată

```
2 + 3
5
+----------------+
| Cafea     x2 24 |
| Croissant x3 21 |
+----------------+
  Total: 45 lei
```

5. Comanda cu prietenii

f-string

În `tipuri_operatori_if.py`: `def comanda_prieteni():`

Cu virgule, spațiile le pune Python. Cu un f-string (litera `f` lipită de ghilimele) scrii tot rândul ca un singur text, iar ce pui între acolade `{ }` se calculează și intră în text.

1. `print("Total:", 3 * 12.5)`, apoi cu f-string: `print(f"Total: {3 * 12.5} lei")`.
2. Scrie exact rândurile `produs = "Cafea"` și `pret = 12.5`. În acolade pui și aceste nume: `{produs}` devine `Cafea`.
3. Cu f-string, `Cafea: 3 x 12.5 = 37.5 lei`, din `{produs}`, `{pret}` și `{3 * pret}`.
4. La fel, rândul pentru `{2 + 3}` prieteni, cu `{5 * pret}` lei.

Ieșire așteptată

```
Total: 37.5
Total: 37.5 lei
Cafea: 3 x 12.5 = 37.5 lei
Cafea pentru 5 prieteni: 62.5 lei
```

Partea 1 — la clasă

Fiecare problemă aduce o noțiune nouă. Urmezi pașii în ordine.

6. Fișa unui produs

tipuri de date

În `tipuri_operatori_if.py`: `def fisa_produs():`

Un produs din stoc are un nume, un preț, un număr de bucăți și o stare (e sau nu în stoc). Programul afișează fiecare valoare împreună cu tipul ei, apoi valoarea totală a stocului.

1. Creează patru variabile: `nume` cu textul `"Cafea boabe"`, `pret` cu `42.5`, `bucati` cu `3`, `in_stoc` cu `True`.
2. Afișează fiecare variabilă urmată de tipul ei: `print(nume, type(nume))`, apoi la fel pentru celelalte trei.
3. Afișează `"Total:"` urmat de `pret * bucati`.

Ieșire așteptată

```
Cafea boabe <class 'str'>
42.5 <class 'float'>
3 <class 'int'>
True <class 'bool'>
Total: 127.5
```

7. Text sau număr?

input și conversii

În `tipuri_operatori_if.py`: `def text_sau_numar():`

Programul citește o notă de la tastatură și arată ce tip are ce ai tastat, înainte și după conversie.

1. Citește nota: `text = input("Nota: ")`.
2. Afișează `"Ai tastat:"`, apoi `text` și `type(text)`.
3. Transformă textul în număr: `nota = int(text)`. Afișează `"Dupa int():"`, `nota` și `type(nota)`.
4. Afișează `"Nota + 1 ca text:"` urmat de `text + "1"`, apoi `"Nota + 1 ca numar:"` urmat de `nota + 1`.

Ieșire așteptată

```
Nota: 9
Ai tastat: 9 <class 'str'>
Dupa int(): 9 <class 'int'>
Nota + 1 ca text: 91
Nota + 1 ca numar: 10
```

8. Paginarea unui raport

operatori aritmetici

În `tipuri_operatori_if.py`: `def paginare():`

Un raport are un număr de rânduri, iar pe o pagină încap un anumit număr de rânduri. Programul afișează câte pagini pline ies, câte rânduri rămân pe ultima pagină și raportul exact, cu zecimale.

1. Citește două numere întregi: `randuri = int(input("Randuri: "))`, apoi `pe_pagina` cu mesajul `"Pe pagina: "`.
2. Afișează paginile pline cu `//`: `print(f"Pagini pline: {randuri // pe_pagina}")`.
3. La fel, afișează `Randuri ramase:` cu `%` și `Pagini cu zecimale:` cu `/`.

Ieșire așteptată

```
Randuri: 137
Pe pagina: 25
Pagini pline: 5
Randuri ramase: 12
Pagini cu zecimale: 5.48
```

```
Randuri: 100
Pe pagina: 25
Pagini pline: 4
Randuri ramase: 0
Pagini cu zecimale: 4.0
```

9. Azi față de ieri

operatori de comparare

În `tipuri_operatori_if.py`: `def temperaturi():`

Programul citește temperatura de azi și de ieri (numere întregi) și răspunde cu `True`/`False` la două întrebări, apoi afișează diferența.

1. Citește `azi` și `ieri` ca întregi, cu mesajele `"Azi: "` și `"Ieri: "`.
2. Afișează `Mai cald decat ieri:` urmat de rezultatul lui `azi > ieri`. Valoarea `True`/`False` o produce comparația, nu o scrii tu.
3. Afișează `Aceeasi temperatura:` cu `==`, apoi `Diferenta:` cu `azi - ieri`.

Ieșire așteptată

```
Azi: 18
Ieri: 15
Mai cald decat ieri: True
Aceeasi temperatura: False
Diferenta: 3
```

```
Azi: 12
Ieri: 12
Mai cald decat ieri: False
Aceeasi temperatura: True
Diferenta: 0
```

10. Promovat sau restanță

operatori logici

În `tipuri_operatori_if.py`: `def promovare():`

Un elev promovează dacă are media cel puțin 5 **și** mai puțin de 40 de absențe. Programul afișează fiecare condiție, rezultatul și opusul lui.

1. Citește `media` cu `float(input("Media: "))` (poate avea zecimale) și `absente` ca întreg, cu mesajul `"Absente: "`.
2. Afișează separat cele două condiții: `media >= 5` și `absente < 40`.
3. Leagă-le cu `and`: `promovat = media >= 5 and absente < 40`. Afișează `Promovat:`.
4. Afișează `Restanta:` cu `not promovat`.

Ieșire așteptată

```
Media: 7.25
Absente: 12
Media cel putin 5: True
Absente sub 40: True
Promovat: True
Restanta: False
```

```
Media: 8.5
Absente: 45
Media cel putin 5: True
Absente sub 40: False
Promovat: False
Restanta: True
```

11. Calificativul

if / elif / else

În `tipuri_operatori_if.py`: `def calificativ():`

Programul citește o medie și afișează calificativul: cel puțin 9 = `Foarte bine`, cel puțin 7 = `Bine`, cel puțin 5 = `Suficient`, altfel `Insuficient`.

1. Citește media cu `float`, mesajul `"Media: "`.
2. Scrie un `if` pentru pragul 9, apoi `elif` pentru 7, `elif` pentru 5 și `else`, în ordinea asta.
3. Fiecare ramură afișează `Calificativ:` urmat de calificativ.

Ieșire așteptată

```
Media: 9
Calificativ: Foarte bine
```

```
Media: 8.5
Calificativ: Bine
```

```
Media: 4.99
Calificativ: Insuficient
```

Partea 2 — de exersat

Aceleași noțiuni, combinate. De aici primești doar cerința.

12. Cifrele unui cod

exersat

În `tipuri_operatori_if.py`: `def cifrele_numarului():`

Codul unui produs e un număr de exact 3 cifre. Afișează cifra sutelor, a zecilor, a unităților și suma lor.

Ieșire așteptată

```
Numar: 472
Sute: 4
Zeci: 7
Unitati: 2
Suma cifrelor: 13
```

```
Numar: 905
Sute: 9
Zeci: 0
Unitati: 5
Suma cifrelor: 14
```

13. Cât a rulat scriptul

exersat

În `tipuri_operatori_if.py`: `def durata():`

Un script de prelucrare a datelor a rulat un număr întreg de secunde. Afișează durata în ore, minute și secunde.

Ieșire așteptată

```
Secunde: 3725
Durata: 1 h 2 min 5 s
```

```
Secunde: 59
Durata: 0 h 0 min 59 s
```

```
Secunde: 7200
Durata: 2 h 0 min 0 s
```

14. Bon cu reducere

exersat

În `tipuri_operatori_if.py`: `def bon_de_plata():`

Se citesc prețul unui produs (poate avea zecimale) și cantitatea. Dacă totalul e cel puțin 500, reducerea e de 20%; dacă e cel puțin 200, de 10%; altfel nu există reducere. Afișează totalul, reducerea și suma de plată.

Ieșire așteptată

```
Pret: 75
Cantitate: 3
Total: 225.0
Reducere 10%: 22.5
De plata: 202.5
```

```
Pret: 125
Cantitate: 4
Total: 500.0
Reducere 20%: 100.0
De plata: 400.0
```

```
Pret: 40
Cantitate: 2
Total: 80.0
Reducere 0%: 0.0
De plata: 80.0
```

15. Citirea unui senzor

exersat

În `tipuri_operatori_if.py`: `def tip_vreme():`

Un senzor trimite o temperatură întreagă. Sub -50 sau peste 60 valoarea e greșită: `Valoare suspecta`. Altfel: sub 0 = `Inghet`, de la 0 la 15 = `Rece`, de la 16 la 25 = `Placut`, peste 25 = `Cald`.

Ieșire așteptată

```
Temperatura: 22
Placut
```

```
Temperatura: -3
Inghet
```

```
Temperatura: 15
Rece
```

```
Temperatura: 75
Valoare suspecta
```

16. Două magazine

exersat

În `tipuri_operatori_if.py`: `def doua_magazine():`

Același produs are în magazinul A un preț și o reducere procentuală, iar în magazinul B alt preț și altă reducere. Afișează prețul final din fiecare magazin și care e mai ieftin, sau `Acelasi pret`.

Ieșire așteptată

```
Pret A: 200
Reducere A: 15
Pret B: 210
Reducere B: 20
Final A: 170.0
Final B: 168.0
Mai ieftin: B
```

```
Pret A: 100
Reducere A: 10
Pret B: 90
Reducere B: 0
Final A: 90.0
Final B: 90.0
Acelasi pret
```

Partea 3 — provocări

17. Restul în bancnote

provocare

În `tipuri_operatori_if.py`: `def rest_in_bancnote():`

Se citesc suma de plată și suma dată (lei întregi). Afișează restul și cum se dă cu cât mai puține bancnote de 50, 10, 5 și 1 leu. Dacă suma dată e mai mică decât cea de plată, afișează `Suma nu ajunge`.

Ieșire așteptată

```
De plata: 63
Ai dat: 200
Rest: 137
50 lei: 2
10 lei: 3
5 lei: 1
1 leu: 2
```

```
De plata: 45
Ai dat: 50
Rest: 5
50 lei: 0
10 lei: 0
5 lei: 1
1 leu: 0
```

```
De plata: 80
Ai dat: 50
Suma nu ajunge
```

18. Media fără extreme

provocare

În `tipuri_operatori_if.py`: `def media_fara_extreme():`

Un juriu dă patru note întregi. Se elimină o singură dată cea mai mare și o singură dată cea mai mică notă; afișează media celor două rămase.

Ieșire așteptată

```
Nota 1: 7
Nota 2: 9
Nota 3: 10
Nota 4: 6
Media fara extreme: 8.0
```

```
Nota 1: 10
Nota 2: 10
Nota 3: 9
Nota 4: 8
Media fara extreme: 9.5
```

19. Câte zile are luna

provocare

În `tipuri_operatori_if.py`: `def zile_in_luna():`

Se citesc luna și anul. Un an e bisect dacă se împarte la 4 și nu la 100, sau dacă se împarte la 400. Afișează câte zile are luna; dacă luna nu e între 1 și 12, afișează `Luna invalida`.

Ieșire așteptată

```
Luna: 2
Anul: 2024
Luna 2 din 2024 are 29 de zile
```

```
Luna: 2
Anul: 1900
Luna 2 din 1900 are 28 de zile
```

```
Luna: 2
Anul: 2000
Luna 2 din 2000 are 29 de zile
```

```
Luna: 9
Anul: 2025
Luna 9 din 2025 are 30 de zile
```

```
Luna: 13
Anul: 2025
Luna invalida
```

20. Variația vânzărilor

provocare

În `tipuri_operatori_if.py`: `def variatie_lunara():`

Se citesc vânzările de luna trecută și de luna aceasta (întregi). Afișează variația procentuală (diferența împărțită la luna trecută, ori 100), apoi `Crestere`, `Scadere` sau `Stagnare`. Dacă luna trecută are 0, afișează `Nu se poate calcula`.

Ieșire așteptată

```
Luna trecuta: 800
Luna aceasta: 1000
Variatie: 25.0%
Crestere
```

```
Luna trecuta: 1000
Luna aceasta: 800
Variatie: -20.0%
Scadere
```

```
Luna trecuta: 500
Luna aceasta: 500
Variatie: 0.0%
Stagnare
```

```
Luna trecuta: 0
Luna aceasta: 50
Nu se poate calcula
```